

Practical-Cryptography: exercises

Sofia Celi

September 2026

1 Lecture 3, 4 & 5

1.1 TLS 1.3

Goal

In the lectures we followed a TLS 1.3 handshake on slides. In this tutorial, you will run one, capture it, decrypt it, and try to break it.

Part 0: Setup

You need OpenSSL $\geq$ “1.1.1” (3.x preferred), Wireshark (or **tshark**), and Python 3 with the **cryptography** package for Exercise 6.

1. Check your version:

```
openssl version
```

On macOS the default **openssl** is LibreSSL, which lacks several options used below. Install OpenSSL 3 (for example with Homebrew) and use that binary instead.

2. Create a working directory and a self-signed certificate for **localhost**:

```
mkdir tlab && cd tlab
openssl req -x509 -newkey ec -pkeyopt ec_paramgen_curve:prime256v1 \
  -nodes -keyout key.pem -out cert.pem -days 30 \
  -subj "/CN=localhost" -addext "subjectAltName=DNS:localhost"
```

Small question: what did this command do?

3. You will use three terminals: one for the server, one for the client, one for the capture. Start a TLS 1.3 server in the first one and leave it running:

```
openssl s_server -accept 4433 -cert cert.pem -key key.pem -tls1_3
```

Part 1: the handshake

Exercise 1: a first connection. Connect to your server, and then to a real website of your choice:

```
openssl s_client -connect localhost:4433 -tls1_3 -CAfile cert.pem
openssl s_client -connect example.com:443 -tls1_3 -servername example.com
```

This is the client session. Type a line of text in the client local session (localhost) and check that it appears on the server. From the client output, report for both connections:

- (a) the protocol version and the cipher suite. What do the three parts of the cipher suite name stand for? What is *not* in the name, compared with a TLS 1.2 suite?
- (b) the key exchange group (**Server Temp Key**) and the signature type (**Peer signature type**). Where were these two negotiated?
- (c) the certificate verification result. Repeat the local connection *without* `-CAfile cert.pem`. What changes? Does client still complete the handshake? What should a real client do?

Exercise 2: capture and decrypt. In the third terminal, capture loopback traffic (the interface is `lo` on Linux and `lo0` on macOS), or capture directly in Wireshark:

```
sudo tcpdump -i lo -w handshake.pcap port 4433 // Linux
sudo tcpdump -i lo0 -w handshake.pcap port 4433 // MacOS
```

Connect again, this time logging the session secrets, send one line of text, and close the client with Ctrl-D:

```
openssl s_client -connect localhost:4433 -tls1_3 -CAfile cert.pem \
-keylogfile keys.txt
```

Stop the capture and open `handshake.pcap` in Wireshark.

- (a) **Before** giving Wireshark the keys: list the TLS records you see, in order. Which handshake messages are readable? What does everything else look like? You will also see a **ChangeCipherSpec** record: what is it doing in a TLS 1.3 handshake?
- (b) In the ClientHello, find: `random`, `legacy_session_id`, `supported_versions`, `supported_groups`, `key_share`, `signature_algorithms`, `cipher_suites` and `server_name`. For how many groups does the client offer support, and for how many does it send a key share? Why the difference?

- (c) Now load the keys (On the “Wireshark” menu: Preferences → Protocols → TLS → (Pre)-Master-Secret log filename, or `tshark -r handshake.pcap -o tls.keylog_file:keys.txt`). List the messages again. Which ones were hidden? Draw the full message flow and mark which key protects each message.
- (d) Open `keys.txt`. Which secrets does it contain, and to which box of the key schedule does each correspond? What is the hexadecimal value in the second column, and why is it a good index for Wireshark to use?
- (e) Everything a passive network observer learns about this connection is in your answer to (a) and (b). Write it down as a list. Which item would matter most if the server were a real website?

Exercise 3: negotiation and HelloRetryRequest. Both `s_client` and `s_server` accept `-groups`, `-ciphersuites` and `-sigalgs`.

- (a) Force ChaCha20-Poly1305 from the client side:

```
openssl s_client -connect localhost:4433 -tls1_3 -CAfile cert.pem \
  -ciphersuites TLS_CHACHA20_POLY1305_SHA256
```

Who makes the final choice of cipher suite, the client or the server?

- (b) Restart the server with `-groups P-384`, and connect with a client that prefers X25519:

```
openssl s_client -connect localhost:4433 -tls1_3 -CAfile cert.pem \
  -groups X25519:P-384 -trace
```

(If your build does not support `-trace`, use `-msg` or a capture.) How many ClientHello messages are sent? Find the HelloRetryRequest. How does it differ from a normal ServerHello? Look at its `random` value, run the command again, and compare. How many round trips did this handshake cost?

- (c) Compare the two ClientHello messages from (b). What changed? Did the client `random` change? Should it?
- (d) Make the negotiation fail: connect with `-groups X25519` only. Which alert do you get, and who sends it?

Exercise 4: randomness and downgrade protection.

- (a) Make three sequential connections and record the client and server `random` values. Is any part of them predictable? Compare with the TLS 1.2 format (Exercise 4b): what was removed in TLS 1.3, and why?

- (b) Restart the server *without* `-tls1_3`, so that it supports both versions, and connect with `-tls1_2 -trace` (mind the underscore). Look at the last 8 bytes of the server `random`. Decode them as ASCII. What is the server telling the client? Why can an attacker who forces the downgrade not remove this marker?
- (c) The same RNG produces the public `random` values and the ephemeral private key. Explain, in two or three sentences, why a client with a weak or backdoored RNG is broken even though the `random` values are public anyway.

Part 2: keys, authentication and the record layer

Exercise 5: authentication.

- (a) In your decrypted capture, find Certificate, CertificateVerify and Finished. Which algorithm signs the CertificateVerify? Over what data is the signature computed? Why would signing only the certificate, or only the key share, not be enough?
- (b) Connect with a hostname check that must fail:

```
openssl s_client -connect localhost:4433 -tls1_3 -CAfile cert.pem \
    -verify_hostname wrong.example -verify_return_error
```

At which point of the handshake does the client abort? Which messages had already been exchanged?

- (c) Create a second key and certificate. Start the server with `-cert cert.pem -key key2.pem` (a key that does not match the certificate). What happens, and where? If the tool refuses to start, explain which handshake message would have caught the mismatch.
- (d) Mutual authentication: restart the server with `-Verify 1 -CAfile client-cert.pem` and give the client a certificate with `-cert` and `-key`. Which new messages appear, in which flight, and under which keys?
- (e) The server may send application data right after its own Finished. List what it does *not* yet know about the client at that point.

Exercise 6: derive the traffic keys yourself. Make a fresh capture with `-ciphersuites TLS_AES_128_GCM_SHA256 -keylogfile keys.txt`, send the line `hello`, and close the connection. Take the value of `CLIENT_TRAFFIC_SECRET_0` from `keys.txt`.

- (a) Implement `HKDF-Expand-Label(secret, label, context, length)` in Python. It is HKDF-Expand (with SHA-256 here) where the `info` input is the structure:

```

uint16 length
uint8 len("tls13 " + label) || "tls13 " + label
uint8 len(context) || context

```

(`cryptography.hazmat.primitives.kdf.hkdf.HKDFExpand` does the expansion for you.)

- (b) Derive `key = HKDF-Expand-Label(secret, "key", "", 16)` and `iv = HKDF-Expand-Label(secret, "iv", "", 12)`.
- (c) In Wireshark, find the first application-data record sent by the client after the handshake and copy its bytes (5-byte header, then ciphertext and tag). Decrypt it with AES-128-GCM: the nonce is the `iv` XORed with the 64-bit record sequence number (here 0), and the additional data is the 5-byte header. What is the last byte of the plaintext, and what is it for?
- (d) Decrypt the *second* client record. What did you have to change? Where is the sequence number in the packet?
- (e) Why does the client Finished not count towards this sequence number?
- (f) Try to decrypt a *server* record with the client key and IV. What happens? Name the two attacks that would be possible if both directions shared one key and IV.
- (g) Optional: derive `finished_key` from `SERVER_HANDSHAKE_TRAFFIC_SECRET` and recompute the server Finished MAC over the transcript hash. Which messages go into the transcript?

Exercise 7: the record layer.

- (a) In an `s_client` session, type a single `K` on a line of its own. This sends a `KeyUpdate`. Use `-trace` or a decrypted capture to see it. What does the message look like to an observer *without* the keys? Are the new keys in your key log file? What happens to the sequence numbers?
- (b) Why does a connection that uses AES-GCM need `KeyUpdate` at all?
- (c) Optional: write a small TCP proxy (about 20 lines of Python) that forwards traffic between client and server but flips one bit in the first application-data record. What does the receiver do? Which alert is sent? Can the attacker learn *why* the record was rejected, and why does that matter? (Think of padding oracles.)
- (d) With the same proxy, swap two records, or drop one. What happens, and which mechanism detects it?

Exercise 8: resumption and 0-RTT. Restart the server with early data enabled, and save a session ticket from a first connection (type a line, wait a second, then close):

```
openssl s_server -accept 4433 -cert cert.pem -key key.pem -tls1_3 \
    -early_data
openssl s_client -connect localhost:4433 -tls1_3 -CAfile cert.pem \
    -sess_out sess.pem
```

- (a) Resume the session with `-sess_in sess.pem`. The client prints `Reused` instead of `New`. Capture this handshake. Which messages from the full handshake are missing? What authenticates the server now?
- (b) Send early data:

```
echo "GET /early" > early.txt
openssl s_client -connect localhost:4433 -tls1_3 -CAfile cert.pem \
    -sess_in sess.pem -early_data early.txt
```

In the capture, where does the early data travel, and under which key? Which secret in the key schedule is it derived from?

- (c) Run exactly the same command a second time. What do the client and the server print? Which defence is this?
- (d) Restart the server adding `-no_anti_replay` and repeat (b) and (c). What changes? Explain why early data has no protection of its own against replay, using what you know about the role of the server `random`.
- (e) Give one HTTP request that would be safe to send as early data and one that would not.
- (f) Does resumed-session early data have forward secrecy? What would an attacker need to compromise, and when?

Exercise 9: what does TLS not hide?

- (a) Using `SSLKEYLOGFILE=keys.txt` (supported by curl, Firefox and Chrome), capture a visit to a real website and decrypt it. Then close the key log and look at the capture as the network sees it. List everything that is still visible.
- (b) Which of these does Encrypted Client Hello address? Which remain?
- (c) Connect this to the first lectures: TLS hides *what* you ask a server from the network, but not from the server. Give one application where that is not enough, and name the primitive that addresses it.

Exercise 10: extensions (pick one).

- (a) **Post-quantum key exchange.** With `OpenSSL ≥ 3.5` on both sides, connect with `-groups X25519MLKEM768`. Compare the sizes of the ClientHello and ServerHello with the X25519 handshake. What is the server’s “key share” now? Why deploy this before quantum computers exist?
- (b) **Survey.** Write a script that connects to 20 popular websites and records version, cipher suite, group and signature algorithm. How many still accept TLS 1.2 or lower? How many negotiate a post-quantum group?

1.2 DiStefano

Goal

Lecture 4 asked what it takes to prove a statement about data received over TLS 1.3 to somebody who was not the TLS client. In this tutorial you will take that design apart: who holds which key and when (DiStefano), how to make it work for more than one request (Surf), why the bytes must be checked against a grammar (Coral), and where the whole approach stops helping. You will reuse the `tlslab` directory, the certificate and the tools from the TLS 1.3 tutorial.

Notation. $\mathcal{C}$ is the client (prover), $\mathcal{V}$ the verifier, $\mathcal{S}$ the server. A superscript marks a share: $k = k^c \oplus k^v$. HS, CHTS, SHTS, dHS, MS, CATS and SATS are the TLS 1.3 secrets as named on the slides; tk denotes a traffic key (key and IV) derived from one of them.

Exercise 0: run it (optional; needs a C++ toolchain). The course repository contains an implementation of the protocol from the slides in `tutorials/surf-impl` and future upcoming work. Its *masked* mode (`SURF_TRUE=0`) is DiStefano as presented in the lecture: every response record is opened through the 2PC keystream circuit, which returns $E_i = \text{AES}_k(ctr_i) \oplus \text{AES}_s(ctr_i)$ to the verifier.

You need CMake, a C++17 compiler, Go and Perl (for BoringSSL), plus Gumbo and nghttp2 (`brew install gumbo-parser nghttp2`, or `apt install libgumbo-dev libnghttp2-dev`). The first build takes a while: BoringSSL is compiled from source.

```
cd tutorials/surf-impl/src
git clone https://github.com/google/benchmark.git
mkdir -p build && cd build
cmake .. -DBENCHMARKING=ON
./DeriveCircuits
cp *.txt ../2pc/key-derivation
```

Run one full handshake and query in masked mode. Both commands are run from `build/`, in two terminals. The first process plays the origin server and the verifier; it prints the command for the second.

```
SURF_TRUE=0 ./E2EBench --is_server --ip 127.0.0.1 --round full \
--handoff /tmp/surf_masked 2>&1 | tee full_server.log
```

```
./E2EBench --ip 127.0.0.1 -p 55392 -v 55391 --round full --handoff /tmp/surf_masked
```

The port might change.

- (a) `DeriveCircuits` wrote a set of Bristol-format circuit files. List them. For three of them, say which step of the TLS 1.3 key schedule or record layer they compute, and which of their inputs are secret shares. The first line of each file gives the number of gates and wires: compare a key-derivation circuit with an AES circuit. Why is HKDF so much more expensive in 2PC than AES?
- (b) From the `TIMINGS` block, separate the *offline* work (circuit preprocessing) from the *online* work. Which single item dominates? Compare with the LAN table from the lecture, and with the time of a plain handshake on your machine (`time openssl s_client -connect ... < /dev/null`).
- (c) From the `DATA` block: how many bytes travel between prover and verifier, and how many between prover and origin? Compare the second number with “SSL handshake has read ... and written ...” from your Lecture 3 traces. What would the first number mean on a mobile connection?
- (d) From the round counts: how many round trips involve the origin, and how many the verifier? Does this agree with your answer to Exercise 2(d)?
- (e) Find the per-record keystream step (`KS_DERIVE`) in the timings. Divide by the size of the response to get a cost per AES block, and extrapolate to the 26 653-byte image from the lecture. How close are you to the number on the slide? What accounts for the difference (hardware, batching, preprocessing)?

Part 1: designated-commitment TLS (DiStefano) or zkTLS

Exercise 1: who knows what, and when? Using the protocol figures from the lecture (and from the DiStefano paper), fill in the table. Write *all*, *share* or *nothing*.

	$\mathcal{C}$	$\mathcal{V}$	$\mathcal{S}$
DHE secret, after ServerHello			
SHTS, before the server flight is forwarded			
SHTS, after $\mathcal{V}$ reveals its share			
server Finished key fk_S , at the end of the handshake			
SATS / tk_{sapp} , during the query phase			
SATS / tk_{sapp} , after the commitment phase			
plaintext of the response			

- (a) $\mathcal{V}$ reveals its share of SHTS to $\mathcal{C}$ in the middle of the handshake. Why is this safe? Which property of the TLS 1.3 key schedule (Lecture 3) makes it safe?
- (b) $\mathcal{C}$ must forward the *encrypted* server flight to $\mathcal{V}$ *before* receiving that share. What could a malicious $\mathcal{C}$ do if the order were reversed?
- (c) The server's certificate is not shown to $\mathcal{V}$. What does $\mathcal{V}$ learn about the server instead, and what is the anonymity set? Give one application where hiding the server matters and one where $\mathcal{V}$ must know exactly which server it was.
- (d) $\mathcal{C}$ and $\mathcal{V}$ send the key share $X_c + Z_v$. Compare with the ClientHello in your trace from Lecture 3: can the server tell that it is talking to two parties? List every observable difference you can think of (bytes on the wire, timing, IP addresses).

Exercise 2: how much time does the 2PC have? To the server, a client that is busy running 2PC with a verifier is just a slow client. `slow.py` performs a normal TLS 1.3 handshake but sleeps before sending the client Finished.

```
openssl s_server -accept 4433 -cert cert.pem -key key.pem -tls1_3 -www
python3 slow.py localhost 4433 5 cert.pem
python3 slow.py example.com 443 5
```

- (a) Increase the delay (5, 10, 20, 40, 80 seconds) against three real websites of your choice. When does each give up? How does the failure show?
- (b) The lecture reports about 6–10s of circuit preprocessing and under 1s of online time for the DiStefano handshake. Given (a), explain why the split between *offline* and *online* work is a functional requirement and not only an optimisation.
- (c) Which of the 2PC steps must happen between receiving the server flight and sending the client Finished, and which can be moved before the ClientHello? Why does the protocol “reverse” the usual order so that a key share exists in advance?
- (d) How many round trips does the *server* see, compared with plain TLS 1.3? Where does the additional latency of a DCTLS handshake go?

Exercise 3: commit, then reveal. Why can $\mathcal{C}$ not simply show $\mathcal{V}$ the ciphertext and, later, the key?

- (a) In Python, encrypt a short JSON document with AES-128 in counter mode under a key k . Decrypt the ciphertext under a different key k' . You get *some* plaintext, and no error. Conclude: what does a counter-mode ciphertext commit to?

- (b) AES-GCM adds a tag, so decryption under k' fails. Is that enough? Read the abstract and introduction of Grubbs, Lu and Ristenpart, *Message franking via committing authenticated encryption* (CRYPTO 2017), and explain in your own words what “AES-GCM is not key-committing” means. Why does it matter here in particular, where $\mathcal{C}$ contributes a share of the key?
- (c) Optional: GHASH is a polynomial over $\text{GF}(2^{128})$ evaluated at $H = \text{AES}_k(0)$. For a one-block message, write the tag equation for two keys k, k' and show that a ciphertext block valid under both can be found by solving one linear equation. Implement it.
- (d) In the 2PC decryption from the lecture, $\mathcal{V}$ does not receive the keystream blocks e_i but $E_i = e_i \oplus b_i$ for masks b_i known to $\mathcal{C}$. What would go wrong if $\mathcal{V}$ received e_i ? What would go wrong if $\mathcal{V}$ received nothing at all?
- (e) “What is this algorithm linear in?” The lecture reports 30.73 minutes to decrypt 26 653 bytes. How many AES blocks is that, and how long per block? Extrapolate to a 2 MB web page. Which resource is the bottleneck: CPU, bandwidth or round trips? How would you find out?

Part 2: from one response to a browsing session (Surf)

DiStefano gives one round: query, response, commit, reveal. After the reveal, $\mathcal{C}$ holds the traffic secrets and nothing sent later on that connection can be attested. Real pages are many requests (`index.html`, then `styles.css`, then a form POST ...) and each request depends on the previous response. Surf (upcoming work) explores how to attest such sessions. The exercises below let you rediscover the design space with tools you already have.

Exercise 4: a key update by hand. Put the logging proxy between client and server, and force AES-128-GCM to keep the numbers small:

```
openssl s_server -accept 4441 -cert cert.pem -key key.pem -tls1_3 \
  -ciphersuites TLS_AES_128_GCM_SHA256
python3 proxy.py 4440 4441 cap
openssl s_client -connect localhost:4440 -tls1_3 -CAfile cert.pem \
  -ciphersuites TLS_AES_128_GCM_SHA256 -keylogfile keys.txt
```

In the client session, type `hello`, then a line containing only `K`, then `world`, then Ctrl-D. The file `cap.c2s` (in the dir where the python files live: you can inspect like `xxd cap.c2s | head -20`) now holds every byte the client sent.

- (a) Split `cap.c2s` into TLS records (5-byte header: type, version, length). Which record is which?
- (b) With your HKDF-Expand-Label from the previous tutorial, decrypt the `hello` record using `CLIENT_TRAFFIC_SECRET_0` and sequence number 0.

- (c) Decrypt the next record with sequence number 1. It is five bytes of handshake message plus the inner content type. Decode it. What does the last byte of the message mean?
- (d) Compute $secret_1 = \text{HKDF-Expand-Label}(secret_0, \text{"traffic upd"}, "", 32)$ and decrypt the `world` record. Which sequence number did you need? Check your $secret_1$ against the key log file.
- (e) Try to decrypt the `world` record with the old key. What happens?
- (f) Given $secret_1$, can you compute $secret_0$? Given $secret_0$, can you compute $secret_1, secret_2, \dots$? Keep your answer for Exercise 5(c).

Exercise 5: what can be revealed, and when? Consider three ways to let $\mathcal{C}$ read a response that was encrypted under a secret-shared key:

per block	decrypt every AES block in 2PC; $\mathcal{C}$ never learns the full key
per session	after committing, $\mathcal{V}$ reveals its share of the session's traffic secret
per round	rotate keys between rounds; after committing to round n , reveal only round n 's key

- (a) For each option give: the 2PC cost as a function of the response size and the number of rounds; how long the browser must wait before it can render anything; and what a malicious $\mathcal{C}$ could forge, if anything, *after* the reveal.
- (b) The *per session* option supports exactly one round. Explain why, in terms of what $\mathcal{C}$ knows after the reveal.
- (c) For the *per round* option, which value may be revealed: $secret_n$, or the pair (key_n, iv_n) derived from it? Use your answer to Exercise 4(f). What does this imply about which derivations must run inside 2PC at every round?
- (d) Responses are encrypted under the *server's* write key. The client can send `KeyUpdate`, but can it make the server rotate? Repeat Exercise 4 with `-trace`, once with `K` and once with `k`. Then type a line in the *server's* terminal. When exactly does the server switch its sending key? What does that mean for where a round can end?
- (e) Draw the message flow of one round of a per-round protocol between $\mathcal{C}$, $\mathcal{V}$ and $\mathcal{S}$: request, response, commitment, `KeyUpdate`, reveal. Two of these steps must not be swapped. Which, and what is the attack if they are?
- (f) HTTP/2 multiplexes many requests over one connection, and responses interleave. What is a "round" now? Propose a rule for when to rotate keys, and name its cost.

Exercise 6: breaking a three-party handshake. A DCTLS protocol is only as sound as the checks $\mathcal{V}$ performs. For each variant below, describe a concrete attack by a malicious $\mathcal{C}$ who wants $\mathcal{V}$ to accept a transcript that the real server never produced, and then give the fix.

- (a) $\mathcal{V}$ helps to derive the handshake keys, but is never bound to the *handshake ciphertexts*: $\mathcal{C}$ may forward any bytes it likes as “the encrypted server flight”.
- (b) $\mathcal{C}$ proves in zero knowledge that it knows a valid signature, under some key from a public set, on a value h . $\mathcal{V}$ accepts h from $\mathcal{C}$ and does not recompute it. (What is h supposed to be? See CertificateVerify in Lecture 3.)
- (c) One fix for both is to let $\mathcal{V}$ see the handshake in the clear, by revealing the full handshake traffic keys to it once the ciphertexts are fixed. What does $\mathcal{V}$ learn that it did not learn before? What is lost compared with the ZKPVS approach? Does it endanger the application keys?

Exercise 7: resumption with three parties. Look at the two NewSessionTicket messages in your TLS 1.3 trace: they carry ticket nonces 0 and 1, and the client printed two different resumption PSKs.

- (a) Write the derivation of the PSK from the resumption master secret and the ticket nonce. In a DCTLS session, which of these values are secret-shared and which are public? What must be computed in 2PC?
- (b) A resumed handshake has no Certificate and no CertificateVerify. What convinces a normal client that it is talking to the same server? What, then, convinces $\mathcal{V}$ in the three-party setting?
- (c) Suppose that at the end of a session $\mathcal{V}$ reveals its share of the PSK to $\mathcal{C}$. Describe how $\mathcal{C}$ could later present a “resumed and attested” session that $\mathcal{V}$ never took part in. Conclude: which secrets may be revealed at the end of a session, and which must stay shared for as long as the ticket lives?
- (d) Resumption with `psk_dhe_ke` runs a fresh (EC)DHE exchange. Explain how this lets a protocol recover after a session whose traffic keys were fully revealed, and compare the cost with opening a new connection from scratch: which messages, proofs and circuits are saved?
- (e) “In TLS 1.3, resumption saves bytes, not round trips.” Justify this using your traces from the previous tutorial. Which bytes matter most for a DCTLS protocol, and why?

Part 3: is the committed byte stream what you think it is? (Coral)

Exercise 8: parse trees on paper. Take the grammar $S \rightarrow (S) \mid SS \mid \epsilon$ from the lecture.

- (a) Draw a parse tree for `()()`, convert it to a left-child right-sibling tree, and number the nodes in depth-first order.
- (b) Run `check_node` by hand for the first six nodes. After each step write down the rule stack, the transition stack, and which leaves have been accumulated.
- (c) The prover supplies the tree; the circuit only *checks* it. Why is this cheaper than parsing inside the circuit? Give another everyday example of a problem where checking an answer is cheaper than computing it.
- (d) The string `()()()` has more than one parse tree under this grammar. Find two. Does ambiguity affect the soundness of the statement “the committed bytes belong to the language”? Does it affect a statement such as “the *second child of the root* is ...”? What does this tell you about the grammars you should use?
- (e) “No key appears twice in an object” is not a context-free property. Argue informally why, and explain how extra state carried through `check_node` could enforce it. What does it cost?

Exercise 9: one byte stream, several meanings. A proof about bytes is only useful if everybody agrees on what the bytes mean.

- (a) Parse each of the following with Python’s `json` module and with `JSON.parse` in Node.js (add any other parser you have: `jq`, `Go`, `Rust serde_json`). Record the value of `age` or `id`, or the error.

```

{"age": 7, "age": 19}
{"age": 7, "a\u0067e": 19}
{"age": 17.999999999999999}
{"age": "19"}
{"age": NaN}
{"age": 00000019}
{"age": 7,}
{"id": 9007199254740993}

```

- (b) For each line on which two parsers disagree, or on which the parsed value differs from what the bytes “say”, describe how a malicious client could exploit a proof system that (i) searches the byte stream for `"age":` and proves a property of the digits that follow, or (ii) proves a property of the value returned by one particular parser.
- (c) Start `reflect.py` and request

```

curl -i 'http://127.0.0.1:8080/profile?email=alice@example.com'
curl -i 'http://127.0.0.1:8080/profile?email=%22%2C%20%22age%22%3A%2019%2C%20%22role%22

```

The server is honest and the TLS channel would be authentic. What does the second response contain? Which of the proof systems in (b) accepts it as evidence that **age** is 19? Which checks rule it out: grammar conformance of the body, the status line, the **Content-Type** header, the position of the key in the tree?

- (d) Conclude: list everything that must be proven about an HTTP response before the statement “the server says the user’s age is at least 18” is justified.

Exercise 10: bytes versus what the user sees. Save the following as `broken.html` and open it in a browser.

```
<table><tr><td>Age:</td><p>17</p><td>21</td></tr></table>
<p>Verified: <b>no<p>yes
```

- (a) Compare the source with the DOM shown in the developer tools. Where did the `<p>17</p>` go? Which elements are bold?
- (b) A proof says: “the response conforms to the HTML grammar and the cell after *Age*: contains 21”. Is that what the user saw? Is this file even in the language of a strict HTML grammar?
- (c) Most real pages are malformed and rely on the browser’s error recovery. What are the options for an attestation system (reject such pages, prove the recovery algorithm, attest the DOM instead of the bytes, ...)? Give one drawback of each.

Part 4: limits

Exercise 11: zero knowledge is not the whole story.

- (a) A site may ask for a proof of “age $\geq t$ ” for any threshold t it likes, and can link the proofs of one user. How many proofs does it need to learn an age between 0 and 120 exactly? Now suppose only $t = 18$ is allowed, but the user proves it every day. What does the site learn on the first day the proof succeeds? Propose two mitigations.
- (b) DCTLS attestations are *designated-verifier*: only $\mathcal{V}$ is convinced. What happens if $\mathcal{V}$ colludes with $\mathcal{C}$? With $\mathcal{S}$? What would it take to convince a third party, and what new trust assumption does each option add?
- (c) The server never agreed to any of this. Give one argument for why that is a feature and one for why it is a problem (think of terms of service, rate limiting, and what the server can detect from Exercise 1(d)).
- (d) Pick one deployment from the lecture (age verification, proof of humanity, proof of purchase). Who issues the underlying fact? Who can be excluded? Who learns that a proof was requested?